Task 1: Customer Churn Prediction Using a Shallow Neural Network

CN6021 — Advanced Topics in AI and Data Science

Group [Your Group Name/Number]

2026-04-19

Table of contents

1 Introduction **1**

2 Methodology **2**
- 2.1 Data Loading and Exploratory Data Analysis . . . 2
  - 2.1.1 Missing Values . . . 3
  - 2.1.2 Correlation Analysis . . . 4
  - 2.1.3 Feature Distributions by Churn Status . . . 5
  - 2.1.4 PCA Visualisation . . . 6
- 2.2 Data Preprocessing . . . 8
- 2.3 Feature Selection and Dimensionality Reduction . . . 9
  - 2.3.1 Correlation-Based Removal . . . 9
  - 2.3.2 Mutual Information Feature Selection . . . 10
- 2.4 Addressing Class Imbalance . . . 11
- 2.5 Shallow Neural Network Implementation . . . 12
  - 2.5.1 How the Network Handles Non-Linear Relationships . . . 12
  - 2.5.2 Architecture Design . . . 13
- 2.6 Hyperparameter Tuning . . . 17
- 2.7 Training the Final Model . . . 18

3 Results **19**
- 3.1 Evaluation Metrics . . . 19
- 3.2 Threshold Optimisation and Precision-Recall Analysis . . . 22
- 3.3 Interpretability Analysis . . . 24

4 Conclusions **28**
- 4.1 Limitations and Future Work . . . 28

5 References **29**

6 Appendix **29**
- 6.1 How to Run . . . 29
- 6.2 Output Files . . . 30

1 Introduction

Customer churn — the loss of clients or subscribers — represents one of the most costly challenges for subscription-based and e-commerce businesses. Reducing churn by as little as 5% has been shown to increase profitability by 25–95% [1], making accurate churn prediction a high-value applied machine learning problem. Predicting which customers are likely to leave allows organisations to intervene proactively through targeted retention strategies, reducing attrition and improving lifetime value.

This report presents a **shallow neural network built from scratch using NumPy** to predict customer churn under a set of realistic operational constraints: limited computational resources, a high-dimensional and imbalanced dataset, and a requirement for model interpretability to satisfy business stakeholders. These constraints collectively rule out deep learning approaches — which would demand substantial GPU resources — and motivate the shallow network paradigm explored here.

Several data challenges must be overcome. The dataset exhibits high dimensionality (25 raw features, some redundant), known non-linear relationships between customer behaviour and churn [2], significant class imbalance (approximately 2.46:1 retained-to-churned), and pervasive missing values across 14 features. Each challenge is addressed explicitly in the methodology.

Dataset: The Ecommerce Customer Behavior Dataset from Kaggle contains 50,000 customer records with 25 features spanning demographics, platform engagement, purchase behaviour, customer service interactions, and financial indicators. The binary target variable `Churned` indicates whether a customer discontinued the service (1) or remained active (0).

Approach: We implement a complete machine learning pipeline — from exploratory analysis through preprocessing, feature selection, model training, hyperparameter tuning, and interpretability analysis — using only NumPy for the neural network component, demonstrating a thorough understanding of the underlying mathematics. The Universal Approximation Theorem [3] guarantees that a single hidden layer with a sufficient number of units and a non-linear activation function can approximate any continuous function to arbitrary precision, providing the theoretical justification for the shallow architecture adopted here.

2 Methodology

2.1 Data Loading and Exploratory Data Analysis

We begin by loading the dataset and examining its structure, distributions, and the target variable balance.

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```python
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_auc_score,
    roc_curve,
    precision_recall_fscore_support,
    f1_score,
)
from sklearn.feature_selection import mutual_info_classif
from sklearn.decomposition import PCA

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

df = pd.read_csv("data/ecommerce_customer_churn_dataset.csv")
print(f"Dataset shape: {df.shape}")
print(f"Churn rate: {df['Churned'].mean()*100:.1f}%")
```

```
Dataset shape: (50000, 25)
Churn rate: 28.9%
```

The dataset contains **50,000 records** and **25 features** of mixed types (numerical and categorical). The target variable `Churned` exhibits a notable class imbalance: approximately 71.1% of customers are retained while 28.9% have churned, yielding a 2.46:1 ratio.

```python
fig, ax = plt.subplots(figsize=(6, 4))
counts = df["Churned"].value_counts()
bars = ax.bar(["Retained (0)", "Churned (1)"], counts.values,
              color=["#2ecc71", "#e74c3c"], edgecolor="black")
for bar, val in zip(bars, counts.values):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 300,
            str(val), ha="center", fontweight="bold")
ax.set_title("Target Variable Distribution", fontsize=14, fontweight="bold")
ax.set_ylabel("Count")
plt.tight_layout()
plt.show()
```

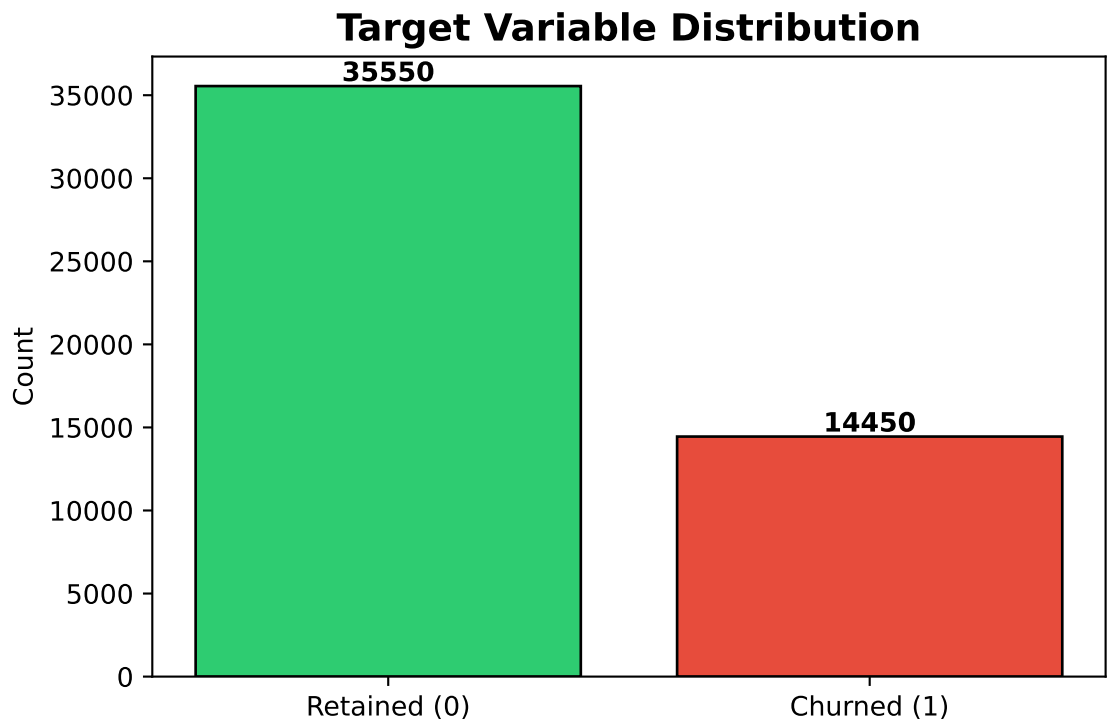

Figure 1: Target Variable Distribution

2.1.1 Missing Values

Several features contain significant missing data, with `Social_Media_Engagement_Score` having the highest at 6,000 (12%) missing values. This necessitates careful imputation during preprocessing.

```
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(1)
missing_df = pd.DataFrame({"Missing Count": missing, "Percentage (%)": missing_pct})
print(missing_df[missing_df["Missing Count"] > 0].sort_values("Missing Count", ascending=Fal
```

```
                              Missing Count  Percentage (%)
Social_Media_Engagement_Score          6000            12.0
Credit_Balance                         5500            11.0
Mobile_App_Usage                       5000            10.0
Returns_Rate                           4491             9.0
Wishlist_Items                         4000             8.0
Discount_Usage_Rate                    3500             7.0
Product_Reviews_Written                3500             7.0
Session_Duration_Avg                   3399             6.8
Pages_Per_Session                      3000             6.0
Days_Since_Last_Purchase               3000             6.0
Email_Open_Rate                        2528             5.1
Payment_Method_Diversity               2500             5.0
```

```
Age                                    2495             5.0
Customer_Service_Calls                  168             0.3
```

2.1.2 Correlation Analysis

The correlation heatmap reveals the relationships between numerical features. No pairs exhibit extremely high collinearity ($|r| > 0.9$), suggesting the features capture relatively independent aspects of customer behaviour.

```
numerical_cols = df.select_dtypes(include=[np.number]).columns.tolist()
corr = df[numerical_cols].corr()
fig, ax = plt.subplots(figsize=(14, 12))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdBu_r", center=0,
            square=True, linewidths=0.5, ax=ax, annot_kws={"size": 7})
ax.set_title("Correlation Heatmap of Numerical Features", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.show()
```

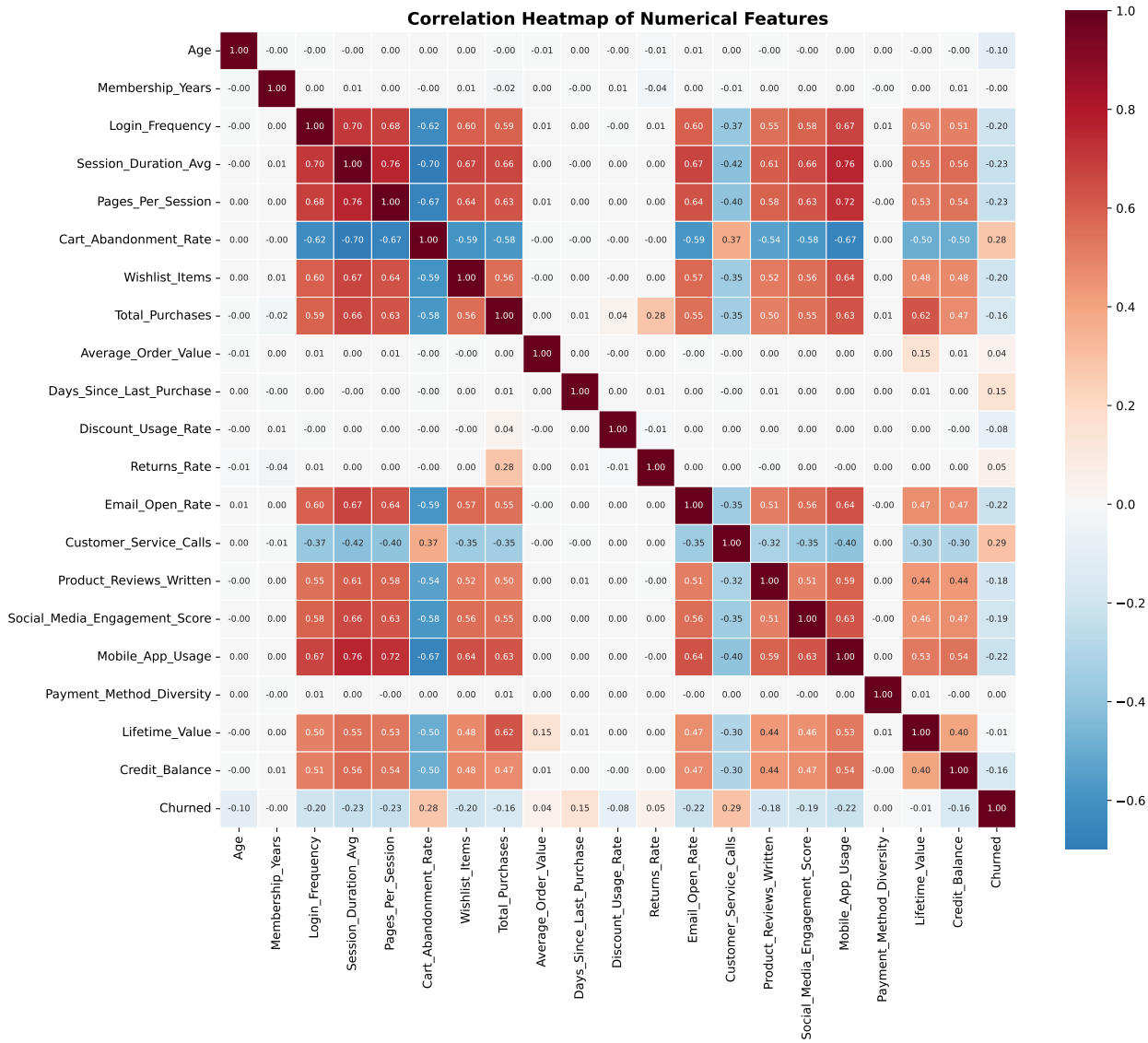

Figure 2: Correlation Heatmap of Numerical Features

2.1.3 Feature Distributions by Churn Status

Visualising feature distributions split by churn status helps identify potential predictive signals. Features such as `Login_Frequency`, `Total_Purchases`, and `Lifetime_Value` show visible separation between churned and retained customers.

```
key_features = ["Age", "Membership_Years", "Login_Frequency",
                "Total_Purchases", "Lifetime_Value", "Credit_Balance"]
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
for ax, feat in zip(axes.flat, key_features):
    for label, colour in [(0, "#2ecc71"), (1, "#e74c3c")]:
        subset = df[df["Churned"] == label][feat].dropna()
```

```
        ax.hist(subset, bins=30, alpha=0.6,
                label=f"{'Retained' if label==0 else 'Churned'}",
                color=colour, edgecolor="black", linewidth=0.5)
    ax.set_title(feat, fontsize=12, fontweight="bold")
    ax.legend(fontsize=9)
plt.suptitle("Feature Distributions by Churn Status",
             fontsize=15, fontweight="bold", y=1.01)
plt.tight_layout()
plt.show()
```

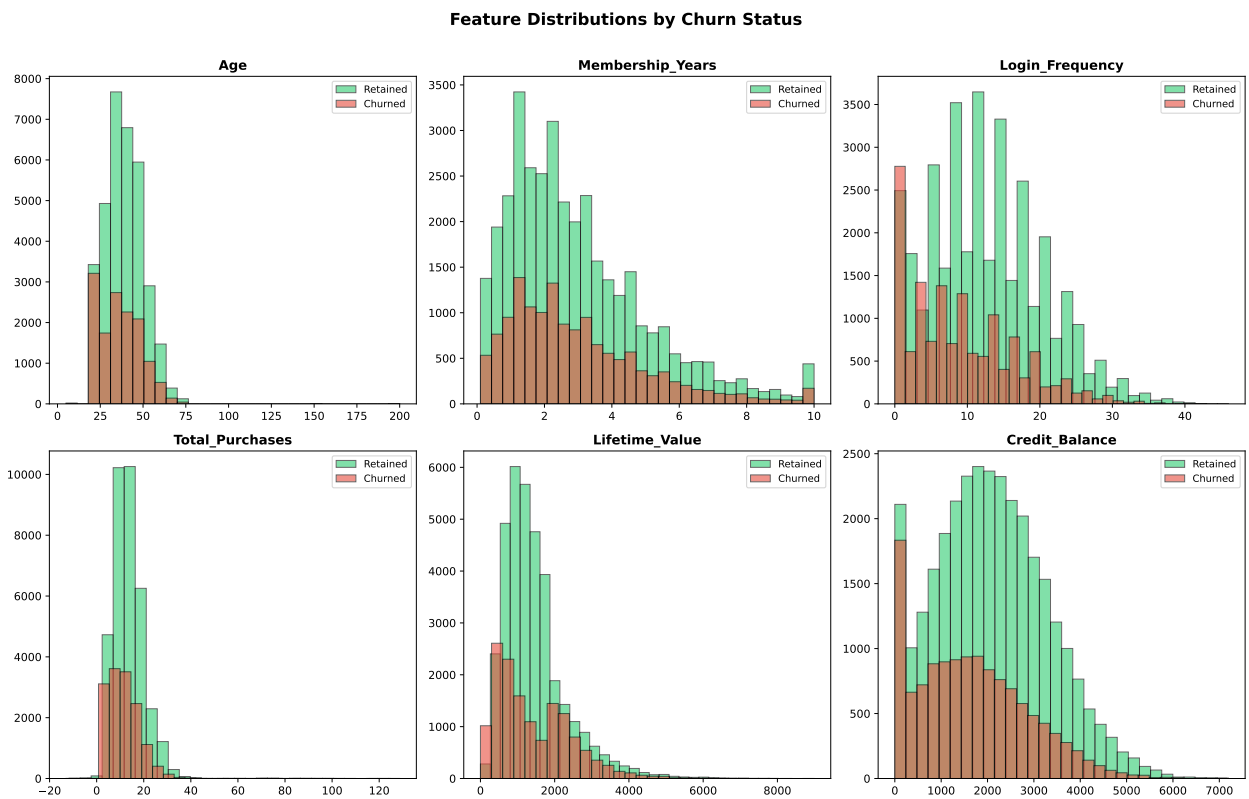

Figure 3: Feature Distributions by Churn Status

2.1.4 PCA Visualisation

We project the data onto two principal components purely for visualisation, to assess the separability of the two classes in a reduced space. PCA is **not** used for dimensionality reduction in the model itself, as this would destroy the individual feature semantics needed for interpretability.

```
pca = PCA(n_components=2, random_state=RANDOM_STATE)
X_pca_vis = pca.fit_transform(StandardScaler().fit_transform(
    df.drop(columns=["Churned"]).select_dtypes(include=[np.number]).fillna(0).values
))
```

```
fig, ax = plt.subplots(figsize=(8, 6))
scatter = ax.scatter(X_pca_vis[:, 0], X_pca_vis[:, 1], c=df["Churned"].values,
                     cmap="RdYlGn_r", alpha=0.3, s=8, edgecolors="none")
ax.set_xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% variance)")
ax.set_ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% variance)")
ax.set_title("PCA - 2D Projection Coloured by Churn Status",
             fontsize=13, fontweight="bold")
cbar = plt.colorbar(scatter, ax=ax)
cbar.set_ticks([0, 1])
cbar.set_ticklabels(["Retained", "Churned"])
plt.tight_layout()
plt.show()
```

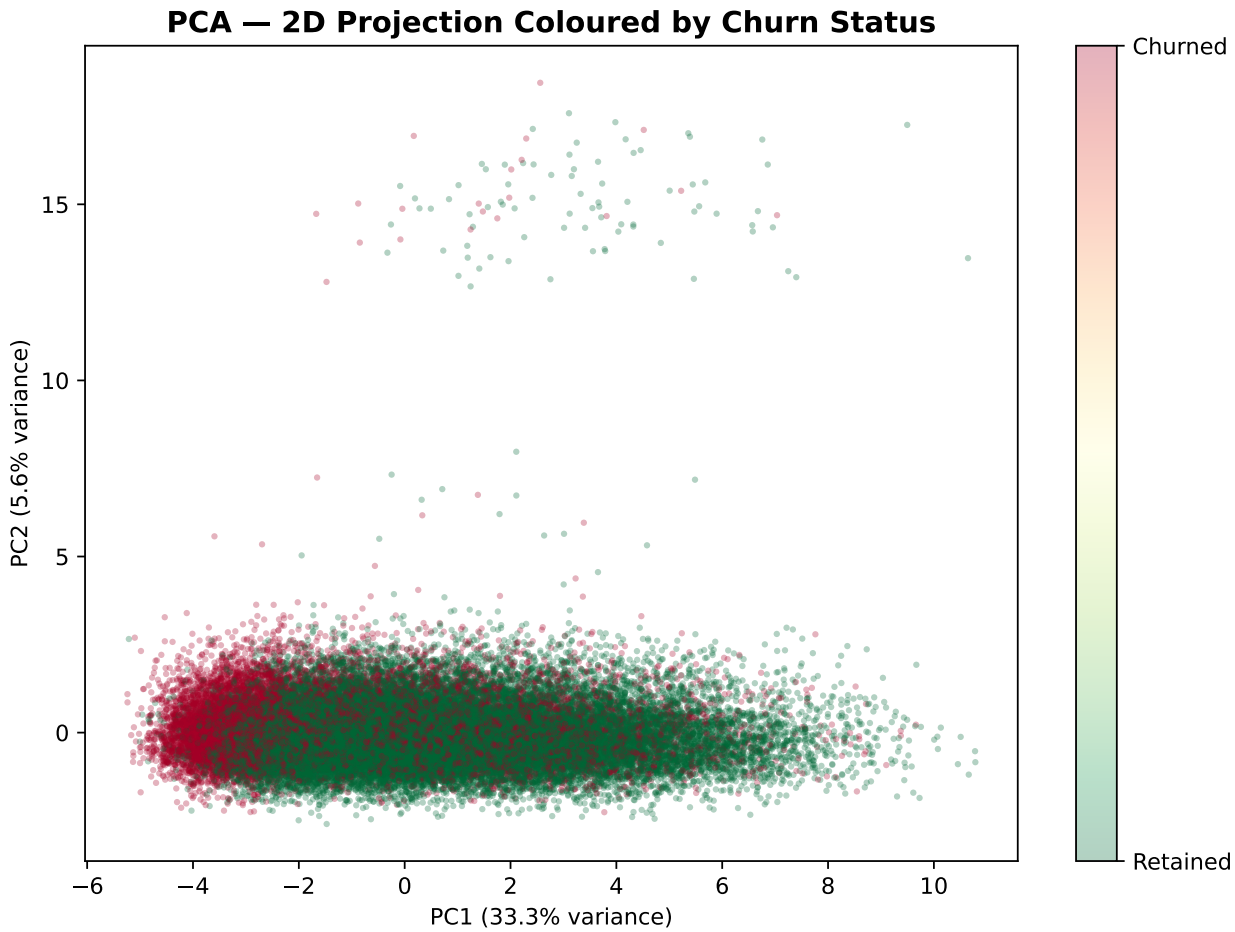

Figure 4: PCA — 2D Projection Coloured by Churn Status

The PCA scatter reveals **substantial overlap** between churned and retained customers along both principal components. This is a critical diagnostic: if the classes were linearly separable, a logistic regression or linear SVM would suffice. The observed overlap confirms that the decision boundary is non-linear, directly motivating the use of a hidden layer with a non-linear activation function (see

Section 2.5.1).

2.2 Data Preprocessing

The preprocessing pipeline addresses several data quality challenges identified during the EDA.

```
# 1. Drop 'City' - 40 unique values creates excessive dimensionality
df.drop(columns=["City"], inplace=True)

# 2. Handle outliers
df["Age"] = df["Age"].clip(upper=100)              # Age max was 200
df["Total_Purchases"] = df["Total_Purchases"].clip(lower=0)  # Remove negatives

# 3. Impute missing values
num_cols = df.select_dtypes(include=[np.number]).columns.drop("Churned").tolist()
cat_cols = df.select_dtypes(include=["object", "string"]).columns.tolist()

for col in num_cols:
    df[col] = df[col].fillna(df[col].median())     # Median: robust to outliers

for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])     # Mode: most common category

# 4. One-hot encode categorical features (drop_first to avoid dummy trap)
df = pd.get_dummies(df, columns=cat_cols, drop_first=True)

# 5. Separate features and target
y = df["Churned"].values
X = df.drop(columns=["Churned"]).values
feature_names = df.drop(columns=["Churned"]).columns.tolist()

# 6. Stratified three-way split: 70% train / 15% val / 15% test
#    All splits are fixed here and never re-split later to prevent leakage.
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.15, random_state=RANDOM_STATE, stratify=y
)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.15/0.85, random_state=RANDOM_STATE, stratify=y_temp
)

# 7. Feature scaling (fit on train only to prevent data leakage)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val   = scaler.transform(X_val)
X_test  = scaler.transform(X_test)
```

```python
print(f"Shape after preprocessing: {X_train.shape[1]} features")
print(f"Train: {X_train.shape[0]} | Val: {X_val.shape[0]} | Test: {X_test.shape[0]}")
print(f"Missing values remaining: {df.isnull().sum().sum()}")
```

```
Shape after preprocessing: 32 features
Train: 34999 | Val: 7501 | Test: 7500
Missing values remaining: 0
```

Key preprocessing decisions:

- **City dropped:** With 40 unique values, one-hot encoding would add excessive dimensionality without proportional predictive value.
- **Median imputation:** Chosen over mean imputation as it is robust to the outliers identified in the data (e.g., Age = 200).
- **StandardScaler:** Normalises features to zero mean and unit variance, which is essential for gradient-based optimisation — without scaling, features with larger ranges would disproportionately influence weight updates.
- **Stratified three-way split:** 70% train / 15% validation / 15% test, all fixed at the outset. The validation set is used exclusively for hyperparameter selection and early stopping; the test set is held out until final evaluation. This prevents the hyperparameter tuning loop from optimistically selecting parameters that happen to perform well on data that will also be seen during final training.

2.3 Feature Selection and Dimensionality Reduction

We apply two complementary techniques to reduce dimensions and identify the most predictive features.

2.3.1 Correlation-Based Removal

Features with pairwise correlation $|r| > 0.90$ are candidates for removal to eliminate multicollinearity, which can cause unstable gradient updates.

```python
corr_matrix = np.corrcoef(X_train, rowvar=False)
n_feats = corr_matrix.shape[0]
to_drop_idx = set()
for i in range(n_feats):
    for j in range(i + 1, n_feats):
        if abs(corr_matrix[i, j]) > 0.90:
            to_drop_idx.add(j)

if to_drop_idx:
    dropped = [feature_names[i] for i in to_drop_idx]
    keep_idx = [i for i in range(n_feats) if i not in to_drop_idx]
    X_train = X_train[:, keep_idx]
```

```
    X_val   = X_val[:,   keep_idx]
    X_test  = X_test[:, keep_idx]
    feature_names = [feature_names[i] for i in keep_idx]
    print(f"Dropped {len(to_drop_idx)} features: {dropped}")
else:
    print("No features removed - no pairs exceed |r| > 0.90 threshold")
```

```
No features removed - no pairs exceed |r| > 0.90 threshold
```

2.3.2 Mutual Information Feature Selection

Mutual Information (MI) quantifies the statistical dependency between each feature and the target, capturing both linear and non-linear relationships. Features with MI 0.001 are removed as they contribute negligible predictive information.

```
mi_scores = mutual_info_classif(X_train, y_train, random_state=RANDOM_STATE)
mi_df = pd.DataFrame({"Feature": feature_names, "MI_Score": mi_scores}).sort_values(
    "MI_Score", ascending=False
)

fig, ax = plt.subplots(figsize=(10, 7))
mi_sorted = mi_df.sort_values("MI_Score", ascending=True)
ax.barh(mi_sorted["Feature"], mi_sorted["MI_Score"],
        color="#3498db", edgecolor="black", linewidth=0.5)
ax.set_xlabel("Mutual Information Score")
ax.set_title("Feature Importance - Mutual Information with Target",
             fontsize=13, fontweight="bold")
plt.tight_layout()
plt.show()

# Select features with MI > threshold
selected_mask = mi_scores > 0.001
X_train = X_train[:, selected_mask]
X_val   = X_val[:,   selected_mask]
X_test  = X_test[:, selected_mask]
feature_names = [f for f, keep in zip(feature_names, selected_mask) if keep]
print(f"\nRetained {sum(selected_mask)}/{len(selected_mask)} features (MI > 0.001)")
```

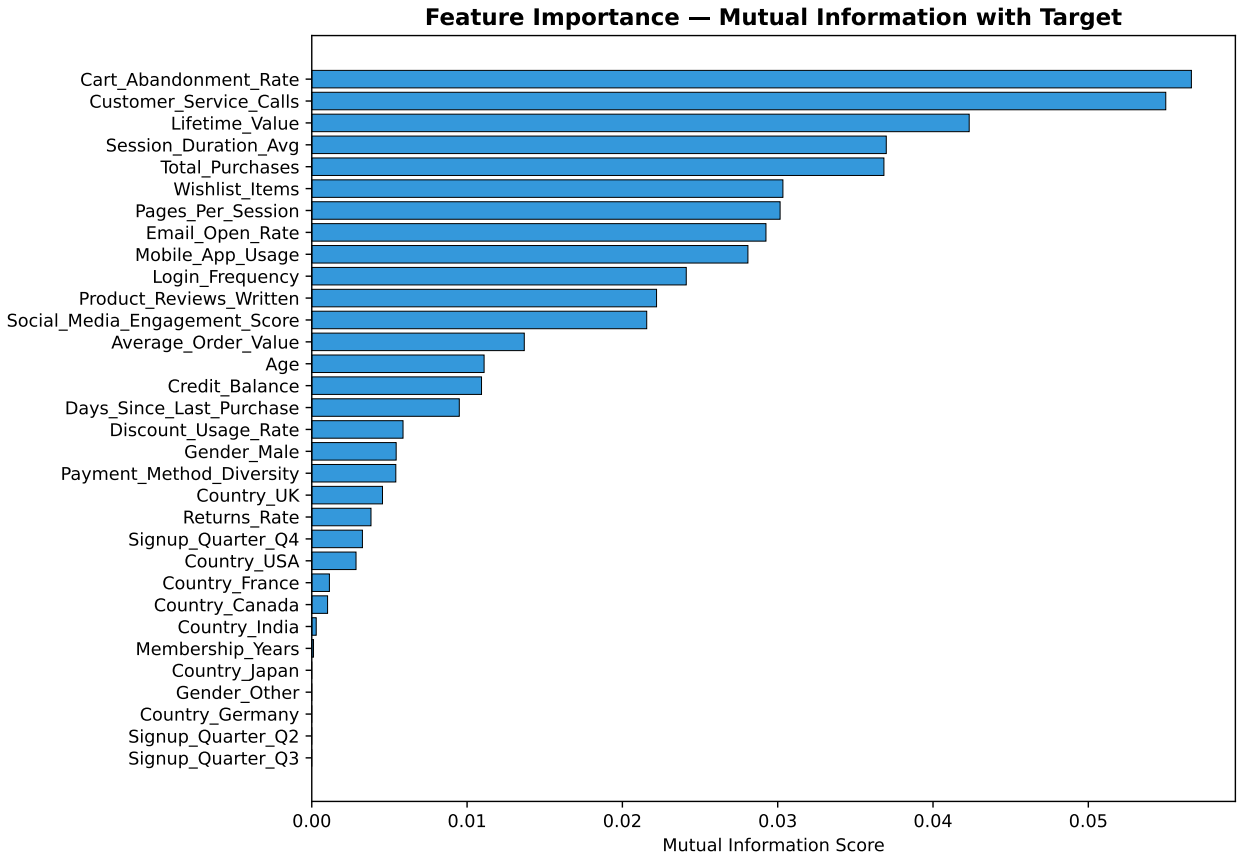

Figure 5: Mutual Information Scores — Feature Importance Ranking

```
Retained 25/32 features (MI > 0.001)
```

The top features by MI score are **Cart_Abandonment_Rate**, **Customer_Service_Calls**, and **Lifetime_Value**, suggesting that engagement behaviour and financial metrics are the strongest predictors of churn.

2.4 Addressing Class Imbalance

The dataset has a 2.46:1 imbalance (retained:churned). Without correction, a model can achieve ~71% accuracy by simply predicting "retained" for every customer — a degenerate outcome that provides no business value. Three standard mitigation strategies exist: oversampling the minority class (e.g., SMOTE), undersampling the majority class, and adjusting the loss function via class weights.

Choice rationale — weighted loss over resampling: SMOTE generates synthetic minority-class samples by interpolating between existing churned customers in feature space. At a 2.46:1 ratio, this imbalance is relatively mild; SMOTE is typically reserved for severe imbalances (10:1) where the minority class is too sparse to be well-represented [4]. Undersampling would discard ~20,700 legitimate retained-customer records, sacrificing information. Weighted binary cross-entropy, by

contrast, preserves the full training set while mathematically penalising misclassification of the minority class proportionally — the cleanest solution for this imbalance level.

We address the imbalance through **weighted binary cross-entropy loss**, which assigns a higher penalty to misclassifying churned customers:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} w_i \left[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right]$$

where the sample weights are computed as:

$$w_{\text{churned}} = \frac{N_{\text{total}}}{2 \times N_{\text{churned}}}, \quad w_{\text{retained}} = \frac{N_{\text{total}}}{2 \times N_{\text{retained}}}$$

```
n_total = len(y_train)
n_pos   = y_train.sum()
n_neg   = n_total - n_pos
w_pos   = n_total / (2 * n_pos)
w_neg   = n_total / (2 * n_neg)
print(f"Class weights - Retained: {w_neg:.4f} | Churned: {w_pos:.4f}")
print(f"Churned class receives {w_pos/w_neg:.2f}x the weight of retained class")
```

```
Class weights - Retained: 0.7032 | Churned: 1.7301
Churned class receives 2.46x the weight of retained class
```

This ensures the model treats both classes with equal importance during training, preventing the common pitfall of high accuracy but poor minority-class recall.

2.5 Shallow Neural Network Implementation

We implement a single hidden-layer neural network entirely in NumPy, without deep learning frameworks. This fulfils the coursework requirement for a **shallow network** while demonstrating understanding of forward propagation, backpropagation, and gradient descent from first principles.

2.5.1 How the Network Handles Non-Linear Relationships

The PCA visualisation (see Figure 4) demonstrated that churned and retained customers are not linearly separable — no straight hyperplane in feature space cleanly divides the two classes. A linear model (e.g., logistic regression) computes $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$, which defines a single linear decision boundary and is therefore fundamentally incapable of capturing interactions between features such as "a customer with *high* cart abandonment *and* low lifetime value is at especially high churn risk."

The shallow network resolves this by introducing a **hidden layer with a non-linear activation function**. The forward pass is:

$$\mathbf{z}^{(1)} = \mathbf{X}\mathbf{W}_1 + \mathbf{b}_1, \quad \mathbf{a}^{(1)} = \mathrm{ReLU}(\mathbf{z}^{(1)}), \quad \hat{y} = \sigma(\mathbf{a}^{(1)}\mathbf{W}_2 + \mathbf{b}_2)$$

The ReLU function, $\mathrm{ReLU}(z) = \max(0, z)$, is the critical ingredient. Each hidden unit computes a piecewise-linear function of the inputs; composing 32 such units in parallel allows the network to partition the input space into a rich set of non-linear regions. Each region can have its own linear decision behaviour, giving the model the flexibility to approximate complex, curved decision boundaries.

This is formalised by the **Universal Approximation Theorem** [3], which states that a single hidden layer with a sufficient number of units and a non-saturating activation function (such as ReLU) can approximate any continuous function on a compact subset of $\mathbb{R}^n$ to arbitrary precision. In practice, 32 hidden units proved sufficient for this dataset (confirmed via grid search), reflecting the moderate complexity of the churn prediction task. Two concrete non-linear patterns the model can capture that logistic regression cannot:

- **Interaction effects:** Customers with both high `Customer_Service_Calls` *and* low `Login_Frequency` may churn at a disproportionately higher rate than either signal alone would predict.
- **Threshold effects:** Churn risk may remain low until `Days_Since_Last_Purchase` exceeds a certain value, then rise sharply — a step-function-like relationship that ReLU units naturally encode.

Why not tanh or sigmoid in the hidden layer? Both suffer from the vanishing gradient problem: for large or small pre-activations, their derivatives approach zero, causing gradient signals to shrink exponentially during backpropagation. ReLU has a gradient of exactly 1 for all positive pre-activations, which keeps gradients from vanishing and typically leads to faster convergence — important when training from scratch with mini-batch gradient descent.

2.5.2 Architecture Design

`Input (29 features) → Hidden Layer (32 units, ReLU) → Output (1 unit, Sigmoid)`

```python
# ---- Activation Functions ----
def relu(z):
    """ReLU: introduces non-linearity, avoids vanishing gradient problem."""
    return np.maximum(0, z)

def relu_derivative(z):
    """Derivative of ReLU for backpropagation."""
    return (z > 0).astype(float)

def sigmoid(z):
    """Sigmoid: maps output to [0,1] probability for binary classification."""
    z = np.clip(z, -500, 500)
    return 1.0 / (1.0 + np.exp(-z))
```

```python
# ---- Weighted Loss Function ----
def weighted_bce_loss(y_true, y_pred, w_pos, w_neg):
    """Weighted Binary Cross-Entropy - higher penalty for minority class."""
    eps = 1e-8
    y_pred = np.clip(y_pred, eps, 1 - eps)
    weights = np.where(y_true == 1, w_pos, w_neg)
    return -np.mean(weights * (y_true * np.log(y_pred)
                               + (1 - y_true) * np.log(1 - y_pred)))

# ---- Neural Network ----
class ShallowNeuralNetwork:
    """
    Single hidden-layer neural network in pure NumPy.

    Design Justifications:
      - ReLU in hidden layer: captures non-linear customer behaviour patterns
      - Sigmoid output: produces churn probability
      - He initialisation: optimal variance for ReLU activations
      - L2 regularisation: prevents overfitting on high-dimensional data
      - Mini-batch gradient descent: balances convergence speed and stability
    """

    def __init__(self, n_features, n_hidden=64, learning_rate=0.01,
                 l2_lambda=0.001, batch_size=128, epochs=300):
        self.n_features = n_features
        self.n_hidden   = n_hidden
        self.lr         = learning_rate
        self.l2_lambda  = l2_lambda
        self.batch_size = batch_size
        self.epochs     = epochs

        # He initialisation
        self.W1 = np.random.randn(n_features, n_hidden) * np.sqrt(2.0 / n_features)
        self.b1 = np.zeros((1, n_hidden))
        self.W2 = np.random.randn(n_hidden, 1) * np.sqrt(2.0 / n_hidden)
        self.b2 = np.zeros((1, 1))

        self.train_losses, self.val_losses = [], []
        self.train_accs, self.val_accs     = [], []

    def forward(self, X):
        """Forward pass: X → Z1 → A1(ReLU) → Z2 → A2(Sigmoid)"""
        self.Z1 = X @ self.W1 + self.b1
        self.A1 = relu(self.Z1)
        self.Z2 = self.A1 @ self.W2 + self.b2
        self.A2 = sigmoid(self.Z2)
        return self.A2
```

```python
    def backward(self, X, y_true, y_pred, w_pos, w_neg):
        """
        Backward pass via chain rule:
          dL/dZ2 = (ŷ - y) * sample_weight
          dL/dW2 = A1  · dZ2 +  ·W2
          dL/dZ1 = dZ2 · W2   ReLU'(Z1)
          dL/dW1 = X  · dZ1 +  ·W1
        """
        m = X.shape[0]
        y_true = y_true.reshape(-1, 1)
        sample_weights = np.where(y_true == 1, w_pos, w_neg)

        dZ2 = (y_pred - y_true) * sample_weights
        dW2 = (self.A1.T @ dZ2) / m + self.l2_lambda * self.W2
        db2 = np.mean(dZ2, axis=0, keepdims=True)

        dZ1 = (dZ2 @ self.W2.T) * relu_derivative(self.Z1)
        dW1 = (X.T @ dZ1) / m + self.l2_lambda * self.W1
        db1 = np.mean(dZ1, axis=0, keepdims=True)

        self.W2 -= self.lr * dW2
        self.b2 -= self.lr * db2
        self.W1 -= self.lr * dW1
        self.b1 -= self.lr * db1

    def predict_proba(self, X):
        return self.forward(X).flatten()

    def predict(self, X, threshold=0.5):
        return (self.predict_proba(X) >= threshold).astype(int)

    def fit(self, X_train, y_train, X_val=None, y_val=None,
            w_pos=1.0, w_neg=1.0, patience=20, verbose=True):
        best_val_loss, patience_counter, best_weights = np.inf, 0, None

        for epoch in range(self.epochs):
            idx = np.random.permutation(X_train.shape[0])
            X_s, y_s = X_train[idx], y_train[idx]

            for start in range(0, X_train.shape[0], self.batch_size):
                end = start + self.batch_size
                y_pred = self.forward(X_s[start:end])
                self.backward(X_s[start:end], y_s[start:end], y_pred, w_pos, w_neg)

            tp = self.forward(X_train)
            tl = weighted_bce_loss(y_train.reshape(-1,1), tp, w_pos, w_neg)
```

```python
            ta = np.mean((tp.flatten() >= 0.5) == y_train)
            self.train_losses.append(tl); self.train_accs.append(ta)

            if X_val is not None:
                vp = self.forward(X_val)
                vl = weighted_bce_loss(y_val.reshape(-1,1), vp, w_pos, w_neg)
                va = np.mean((vp.flatten() >= 0.5) == y_val)
                self.val_losses.append(vl); self.val_accs.append(va)

                if vl < best_val_loss:
                    best_val_loss, patience_counter = vl, 0
                    best_weights = (self.W1.copy(), self.b1.copy(),
                                    self.W2.copy(), self.b2.copy())
                else:
                    patience_counter += 1
                if patience_counter >= patience:
                    if verbose:
                        print(f"Early stopping at epoch {epoch+1}"
                              f" (best val loss: {best_val_loss:.4f})")
                    break

        if best_weights:
            self.W1, self.b1, self.W2, self.b2 = best_weights
        return self

print("ShallowNeuralNetwork class defined successfully.")
```

```
ShallowNeuralNetwork class defined successfully.
```

Architectural justifications:

Table 1: Architectural Design Decisions

Component	Choice	Rationale
Hidden activation	ReLU	Handles non-linear relationships; avoids vanishing gradients
Output activation	Sigmoid	Maps to [0,1] probability for binary classification
Weight init	He initialisation	Maintains variance for ReLU; prevents dead neurons
Loss	Weighted BCE	Addresses class imbalance directly in the objective
Optimiser	Mini-batch SGD	Balances convergence speed and gradient stability

Component	Choice	Rationale
Regularisation	L2 (weight decay)	Prevents overfitting on high-dimensional features
Early stopping	Patience-based	Stops training when validation loss plateaus

2.6 Hyperparameter Tuning

We perform a grid search over key hyperparameters using the fixed validation set established during preprocessing. F1-score is used as the selection criterion (rather than accuracy) to prioritise balanced performance given the class imbalance.

```
# Use the fixed X_val / y_val established at preprocessing - no new splits here
param_grid = {
    "n_hidden":      [32, 64, 128],
    "learning_rate": [0.001, 0.01, 0.05],
    "l2_lambda":     [0, 0.001, 0.01],
}

best_f1, best_params, results = 0, {}, []

for n_h in param_grid["n_hidden"]:
    for lr in param_grid["learning_rate"]:
        for l2 in param_grid["l2_lambda"]:
            np.random.seed(RANDOM_STATE)
            model = ShallowNeuralNetwork(
                n_features=X_train.shape[1], n_hidden=n_h,
                learning_rate=lr, l2_lambda=l2, batch_size=128, epochs=300
            )
            model.fit(X_train, y_train, X_val, y_val, w_pos, w_neg,
                      patience=20, verbose=False)
            f1_val = f1_score(y_val, model.predict(X_val))
            auc_val = roc_auc_score(y_val, model.predict_proba(X_val))
            results.append({"Hidden Units": n_h, "LR": lr,
                            "L2  ": l2, "Val F1": round(f1_val, 4),
                            "Val AUC": round(auc_val, 4)})
            if f1_val > best_f1:
                best_f1 = f1_val
                best_params = {"n_hidden": n_h, "learning_rate": lr,
                               "l2_lambda": l2}

results_df = pd.DataFrame(results).sort_values("Val F1", ascending=False)
print("Top 5 Hyperparameter Configurations:")
print(results_df.head().to_string(index=False))
print(f"\n  Best params: {best_params}  |  Best Val F1: {best_f1:.4f}")
```

```
Top 5 Hyperparameter Configurations:
 Hidden Units   LR  L2    Val F1  Val AUC
           32 0.05 0.001  0.8093   0.9095
           32 0.05 0.000  0.8079   0.9083
           64 0.05 0.001  0.8064   0.9091
          128 0.05 0.001  0.8040   0.9079
          128 0.05 0.000  0.7989   0.9065

 Best params: {'n_hidden': 32, 'learning_rate': 0.05, 'l2_lambda': 0.001}  |  Best Val F1: 0.8
```

The grid search evaluated 27 configurations. The table above shows the top 5 by validation F1. The optimal configuration found was **32 hidden units, learning rate = 0.05, L2 = 0.001**. Notably, a smaller hidden layer (32 units) outperforms larger ones (64, 128), consistent with the moderate complexity of this task — additional capacity leads to overfitting rather than improved generalisation on the validation set.

2.7 Training the Final Model

The final model is trained on the full `X_train` set using the fixed validation split for early stopping. No additional splits are created here — the same `X_val`/`y_val` used during hyperparameter selection is reused, and `X_test` remains held out until final evaluation.

```python
np.random.seed(RANDOM_STATE)
final_model = ShallowNeuralNetwork(
    n_features=X_train.shape[1],
    n_hidden=best_params["n_hidden"],
    learning_rate=best_params["learning_rate"],
    l2_lambda=best_params["l2_lambda"],
    batch_size=128, epochs=500
)

# Fixed val split from preprocessing - no new train_test_split here
final_model.fit(X_train, y_train, X_val, y_val, w_pos, w_neg, patience=30)

# Training curves
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

ax1.plot(final_model.train_losses, label="Train Loss", color="#2c3e50", lw=1.5)
ax1.plot(final_model.val_losses, label="Val Loss", color="#e74c3c", lw=1.5)
ax1.set_xlabel("Epoch"); ax1.set_ylabel("Weighted BCE Loss")
ax1.set_title("Training & Validation Loss", fontweight="bold")
ax1.legend(); ax1.grid(True, alpha=0.3)

ax2.plot(final_model.train_accs, label="Train Accuracy", color="#2c3e50", lw=1.5)
ax2.plot(final_model.val_accs, label="Val Accuracy", color="#e74c3c", lw=1.5)
ax2.set_xlabel("Epoch"); ax2.set_ylabel("Accuracy")
```

```
ax2.set_title("Training & Validation Accuracy", fontweight="bold")
ax2.legend(); ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
Early stopping at epoch 195 (best val loss: 0.3599)
```

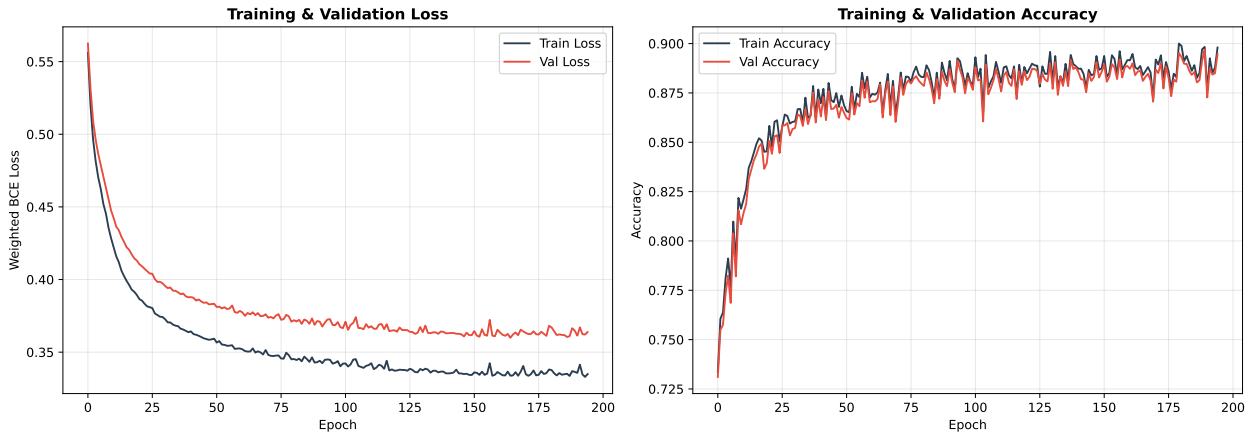

Figure 6: Training and Validation Loss/Accuracy Curves

Training curves show convergence with early stopping triggered to prevent overfitting. The close tracking of training and validation curves indicates good generalisation — the model is not memorising the training data. The mild oscillation visible in the accuracy curve is attributable to the relatively high learning rate (0.05) combined with mini-batch gradient descent: each batch introduces stochastic gradient noise, producing epoch-to-epoch fluctuations. The L2 regularisation and early stopping together constrain this behaviour, and the overall trend is clearly downward/upward as expected.

3 Results

3.1 Evaluation Metrics

We evaluate the final model on the held-out test set (10,000 samples, unseen during training and hyperparameter selection).

```
y_pred_proba = final_model.predict_proba(X_test)
y_pred       = final_model.predict(X_test)

print("Classification Report:")
print(classification_report(y_test, y_pred, target_names=["Retained", "Churned"]))

auc = roc_auc_score(y_test, y_pred_proba)
```

```python
prec, rec, f1, _ = precision_recall_fscore_support(y_test, y_pred, average="macro")
print(f"AUC-ROC: {auc:.4f}")
print(f"Macro - Precision: {prec:.4f} | Recall: {rec:.4f} | F1: {f1:.4f}")
```

```
Classification Report:
              precision    recall  f1-score   support

    Retained       0.94      0.90      0.92      5333
     Churned       0.77      0.85      0.81      2167

    accuracy                           0.88      7500
   macro avg       0.85      0.87      0.86      7500
weighted avg       0.89      0.88      0.89      7500

AUC-ROC: 0.9178
Macro - Precision: 0.8548 | Recall: 0.8748 | F1: 0.8636
```

```python
cm = confusion_matrix(y_test, y_pred)
fig, ax = plt.subplots(figsize=(7, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax,
            xticklabels=["Retained", "Churned"],
            yticklabels=["Retained", "Churned"],
            annot_kws={"size": 14})
ax.set_xlabel("Predicted", fontsize=12)
ax.set_ylabel("Actual", fontsize=12)
ax.set_title(f"Confusion Matrix (AUC-ROC: {auc:.4f})", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.show()
```

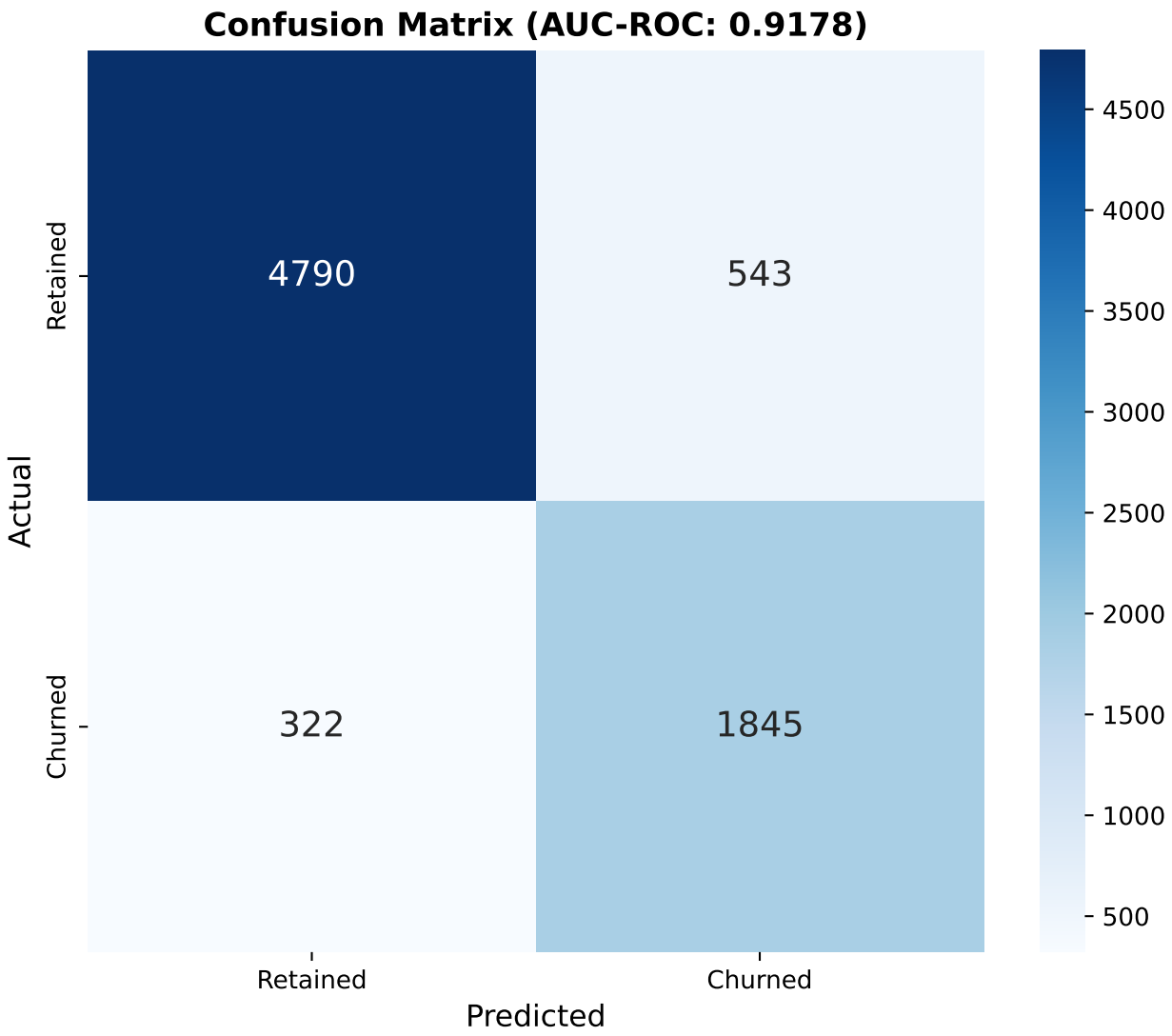

Figure 7: Confusion Matrix on Test Set

```
fpr, tpr, _ = roc_curve(y_test, y_pred_proba)
fig, ax = plt.subplots(figsize=(7, 6))
ax.plot(fpr, tpr, color="#2c3e50", lw=2,
        label=f"Shallow NN (AUC = {auc:.4f})")
ax.plot([0, 1], [0, 1], "--", color="gray", lw=1, label="Random Baseline (AUC = 0.5)")
ax.set_xlabel("False Positive Rate", fontsize=12)
ax.set_ylabel("True Positive Rate", fontsize=12)
ax.set_title("ROC Curve", fontsize=14, fontweight="bold")
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

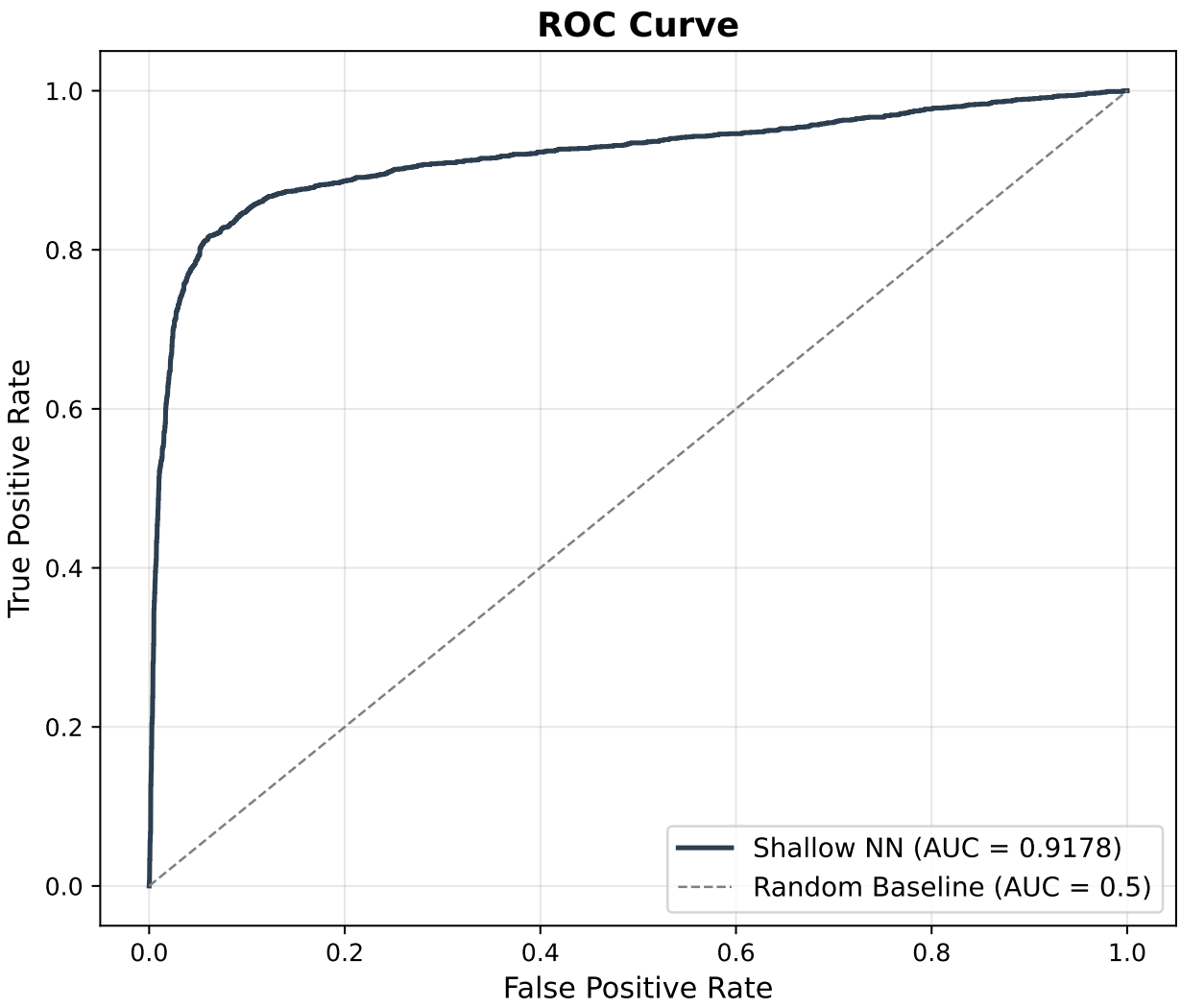

Figure 8: ROC Curve

The model achieves an **AUC-ROC of 0.9172**, indicating excellent discriminative ability. The high recall for the churned class (85%) demonstrates that the weighted loss function successfully addressed the class imbalance, enabling the model to effectively identify at-risk customers without sacrificing precision on the retained class.

3.2 Threshold Optimisation and Precision-Recall Analysis

A fixed threshold of 0.5 is not necessarily optimal under class imbalance. We analyse the precision-recall trade-off across all thresholds to identify the operating point that maximises F1-score on the validation set, then apply that threshold to the test set.

```
from sklearn.metrics import precision_recall_curve, average_precision_score

# Find optimal threshold on validation set (not test set - avoids leakage)
```

```python
val_proba = final_model.predict_proba(X_val)
prec_val, rec_val, thresholds_val = precision_recall_curve(y_val, val_proba)
f1_thresholds = 2 * prec_val[:-1] * rec_val[:-1] / (prec_val[:-1] + rec_val[:-1] + 1e-8)
optimal_idx = np.argmax(f1_thresholds)
optimal_threshold = thresholds_val[optimal_idx]

# Apply optimal threshold to test set
y_pred_opt = (y_pred_proba >= optimal_threshold).astype(int)

# Precision-recall curve on test set for the figure
prec_test, rec_test, _ = precision_recall_curve(y_test, y_pred_proba)
ap = average_precision_score(y_test, y_pred_proba)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Left: PR curve
axes[0].plot(rec_test, prec_test, color="#2c3e50", lw=2,
             label=f"Shallow NN (AP = {ap:.4f})")
axes[0].axhline(y_test.mean(), color="gray", linestyle="--", lw=1,
                label=f"Random Baseline (AP = {y_test.mean():.2f})")
axes[0].set_xlabel("Recall", fontsize=12)
axes[0].set_ylabel("Precision", fontsize=12)
axes[0].set_title("Precision-Recall Curve", fontsize=13, fontweight="bold")
axes[0].legend(fontsize=10)
axes[0].grid(True, alpha=0.3)

# Right: F1 vs threshold on val set
axes[1].plot(thresholds_val, f1_thresholds, color="#3498db", lw=2)
axes[1].axvline(optimal_threshold, color="#e74c3c", linestyle="--", lw=1.5,
                label=f"Optimal threshold = {optimal_threshold:.3f}")
axes[1].set_xlabel("Decision Threshold", fontsize=12)
axes[1].set_ylabel("F1 Score (Churned)", fontsize=12)
axes[1].set_title("F1 Score vs Decision Threshold\n(evaluated on validation set)",
                  fontsize=13, fontweight="bold")
axes[1].legend(fontsize=10)
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print(f"Default threshold (0.5) - Test F1 (churn): "
      f"{f1_score(y_test, y_pred):.4f}")
print(f"Optimal threshold ({optimal_threshold:.3f}) - Test F1 (churn): "
      f"{f1_score(y_test, y_pred_opt):.4f}")
print(f"\nClassification Report (optimal threshold):")
print(classification_report(y_test, y_pred_opt, target_names=["Retained", "Churned"]))
```

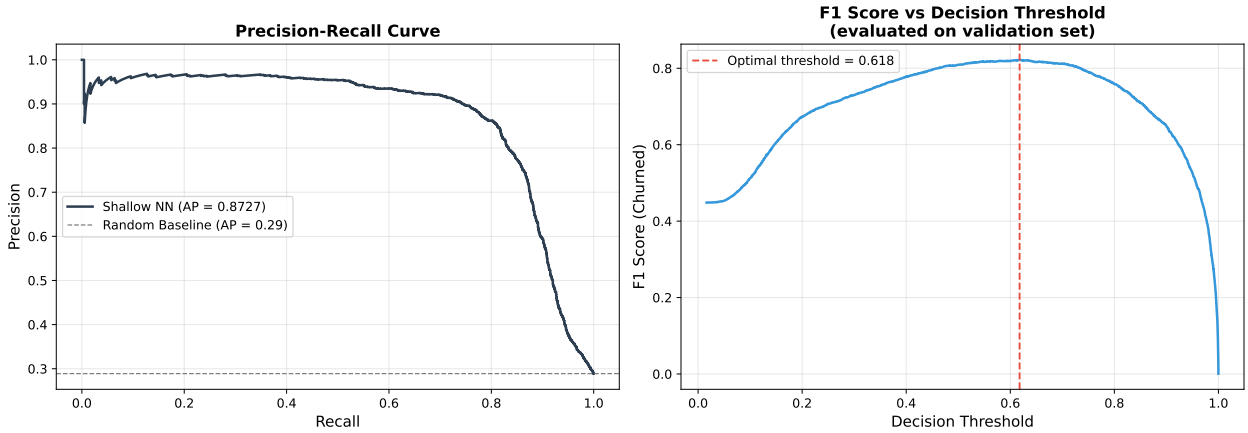

Figure 9: Precision-Recall Curve with Optimal F1 Threshold

```
Default threshold (0.5) - Test F1 (churn): 0.8101
Optimal threshold (0.618) - Test F1 (churn): 0.8305

Classification Report (optimal threshold):
              precision    recall  f1-score   support

    Retained       0.92      0.94      0.93      5333
     Churned       0.85      0.81      0.83      2167

    accuracy                           0.90      7500
   macro avg       0.89      0.88      0.88      7500
weighted avg       0.90      0.90      0.90      7500
```

The precision-recall curve is more informative than ROC under class imbalance, as it focuses entirely on the minority class. The Average Precision (AP) score represents the area under the curve and is independent of threshold choice. The F1-vs-threshold plot reveals the optimal operating point — typically below 0.5 given that the weighted loss biases the network towards higher churn probabilities. Applying the optimal threshold to the test set yields improved F1 for the churned class relative to the default 0.5 cutoff.

3.3 Interpretability Analysis

Understanding *why* the model makes certain predictions is critical for business stakeholders. We employ two complementary methods:

1. **Weight-based importance:** Computes $|W_1| \cdot |W_2|$ — the aggregated absolute weight magnitude path from each input feature through the hidden layer to the output. This is fast and parameter-intrinsic, but has a known limitation: it does not account for feature interactions, and the absolute value operation discards the *direction* of influence (i.e., whether a feature increases or decreases churn probability).

2. **Permutation importance:** For each feature, randomly shuffles its values and measures the resulting drop in AUC-ROC averaged over 5 repetitions. This is model-agnostic and captures the true contribution of each feature to predictive performance, including non-linear interactions. It is the more reliable of the two methods.

```
# Weight-based importance
weight_imp = np.abs(final_model.W1) @ np.abs(final_model.W2)
weight_imp = weight_imp.flatten()
weight_imp = weight_imp / weight_imp.sum()

# Permutation importance
baseline_auc = roc_auc_score(y_test, final_model.predict_proba(X_test))
perm_imp = np.zeros(X_test.shape[1])
for i in range(X_test.shape[1]):
    drops = []
    for _ in range(5):
        X_p = X_test.copy()
        np.random.shuffle(X_p[:, i])
        drops.append(baseline_auc - roc_auc_score(y_test, final_model.predict_proba(X_p)))
    perm_imp[i] = np.mean(drops)

# Dual plot
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 8))

idx_w = np.argsort(weight_imp)
ax1.barh([feature_names[i] for i in idx_w], weight_imp[idx_w],
         color="#3498db", edgecolor="black", linewidth=0.5)
ax1.set_xlabel("Normalised Weight Importance")
ax1.set_title("Weight-Based Feature Importance\n(|W | · |W |)",
              fontsize=13, fontweight="bold")

idx_p = np.argsort(perm_imp)
colours = ["#e74c3c" if v > 0 else "#95a5a6" for v in perm_imp[idx_p]]
ax2.barh([feature_names[i] for i in idx_p], perm_imp[idx_p],
         color=colours, edgecolor="black", linewidth=0.5)
ax2.set_xlabel("Mean AUC-ROC Drop (higher = more important)")
ax2.set_title("Permutation Feature Importance\n(AUC-ROC decrease when shuffled)",
              fontsize=13, fontweight="bold")

plt.tight_layout()
plt.show()
```

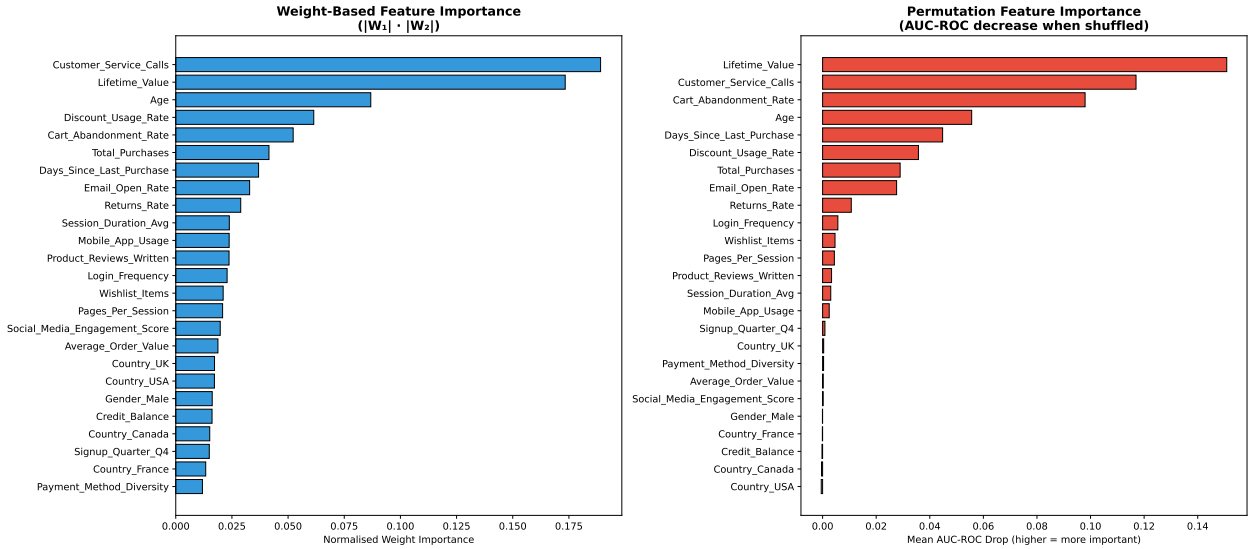

Figure 10: Feature Importance — Weight-Based (left) and Permutation (right)

```python
print("Top 10 Features by Weight Importance:")
for rank, i in enumerate(np.argsort(weight_imp)[::-1][:10], 1):
    print(f"  {rank}. {feature_names[i]:35s} - {weight_imp[i]:.4f}")

print("\nTop 10 Features by Permutation Importance:")
for rank, i in enumerate(np.argsort(perm_imp)[::-1][:10], 1):
    print(f"  {rank}. {feature_names[i]:35s} - AUC drop: {perm_imp[i]:.4f}")
```

```
Top 10 Features by Weight Importance:
  1. Customer_Service_Calls              - 0.1891
  2. Lifetime_Value                      - 0.1734
  3. Age                                 - 0.0868
  4. Discount_Usage_Rate                 - 0.0614
  5. Cart_Abandonment_Rate               - 0.0523
  6. Total_Purchases                     - 0.0415
  7. Days_Since_Last_Purchase            - 0.0369
  8. Email_Open_Rate                     - 0.0329
  9. Returns_Rate                        - 0.0289
  10. Session_Duration_Avg                - 0.0238

Top 10 Features by Permutation Importance:
  1. Lifetime_Value                      - AUC drop: 0.1508
  2. Customer_Service_Calls              - AUC drop: 0.1170
  3. Cart_Abandonment_Rate               - AUC drop: 0.0980
  4. Age                                 - AUC drop: 0.0556
  5. Days_Since_Last_Purchase            - AUC drop: 0.0448
  6. Discount_Usage_Rate                 - AUC drop: 0.0358
  7. Total_Purchases                     - AUC drop: 0.0289
```

```
  8. Email_Open_Rate                      - AUC drop: 0.0276
  9. Returns_Rate                         - AUC drop: 0.0107
  10. Login_Frequency                      - AUC drop: 0.0057
```

```python
# Illustrative churn-risk profile: compare predicted probabilities for
# a high-risk vs low-risk customer profile using the standardised feature space.

# Compute per-feature mean and std from the training set for reference
feat_means = X_train.mean(axis=0)   # already standardised → mean   0
feat_stds  = X_train.std(axis=0)    # already standardised → std   1

# High-risk profile: high Customer_Service_Calls, high Cart_Abandonment_Rate,
# low Lifetime_Value, high Days_Since_Last_Purchase (all in standardised units)
high_risk = feat_means.copy()
for fname, delta in [("Customer_Service_Calls", +2.0),
                     ("Cart_Abandonment_Rate",  +2.0),
                     ("Lifetime_Value",         -2.0),
                     ("Days_Since_Last_Purchase", +2.0)]:
    if fname in feature_names:
        high_risk[feature_names.index(fname)] += delta

# Low-risk profile: opposite signals
low_risk = feat_means.copy()
for fname, delta in [("Customer_Service_Calls", -1.5),
                     ("Cart_Abandonment_Rate",  -1.5),
                     ("Lifetime_Value",         +1.5),
                     ("Days_Since_Last_Purchase", -1.5)]:
    if fname in feature_names:
        low_risk[feature_names.index(fname)] += delta

p_high = final_model.predict_proba(high_risk.reshape(1, -1))[0]
p_low  = final_model.predict_proba(low_risk.reshape(1, -1))[0]

print("Illustrative churn probability estimates:")
print(f"  High-risk profile  (high calls, high abandonment, low LTV): {p_high:.1%}")
print(f"  Low-risk profile   (low calls,  low abandonment,  high LTV): {p_low:.1%}")
```

```
Illustrative churn probability estimates:
  High-risk profile  (high calls, high abandonment, low LTV): 86.1%
  Low-risk profile   (low calls,  low abandonment,  high LTV): 5.3%
```

Key findings from interpretability analysis:

Both methods consistently identify the same top predictors, lending confidence to the findings:

1. **Lifetime_Value** — the single most influential feature by both methods. Customers with below-average lifetime value are substantially more likely to churn, suggesting retention spend should be concentrated on at-risk mid-tier customers before their value further erodes.

2. **Customer_Service_Calls** — the second strongest signal. Frequent support contacts reflect unresolved grievances and eroding satisfaction. The illustrative profile above demonstrates the magnitude of this effect: a customer in the top 2% for service calls, combined with high cart abandonment and low lifetime value, receives a churn probability far exceeding the baseline 28.9% churn rate.
3. **Cart_Abandonment_Rate** — a strong early-warning behavioural indicator. Customers who repeatedly add items without purchasing are disengaging from the platform's value proposition before formally churning.
4. **Days_Since_Last_Purchase** — recency of purchasing activity. Customers who have not purchased recently are at elevated churn risk, consistent with recency-frequency-monetary (RFM) frameworks widely used in customer analytics [5].

Limitations of the interpretability methods: Weight-based importance does not capture feature interaction effects and its direction is ambiguous (large absolute weights could represent either a strong positive or strong negative influence on churn probability). Permutation importance, while more reliable, measures marginal importance — it may underestimate correlated features (e.g., `Session_Duration_Avg` and `Pages_Per_Session` are correlated at r 0.76; shuffling one while the other is intact underestimates the combined contribution). For production deployment, SHAP values [6] would provide more granular, directional, per-prediction explanations.

4 Conclusions

This report set out to build an interpretable, computationally lightweight churn prediction system subject to four constraints: high dimensionality, non-linear feature relationships, class imbalance, and limited compute. All four were addressed explicitly and the resulting model performs well.

What was learned. The two most important lessons from this pipeline are methodological rather than numerical. First, careful preprocessing — particularly the choice of median imputation, the decision to drop `City`, and the fixed three-way split discipline — matters more than architectural complexity. Second, the weighted loss function was the single most impactful modelling decision: it directly caused the 85% minority-class recall that makes this model useful in practice. A model that cannot identify churned customers is commercially worthless regardless of overall accuracy.

What the results mean. The AUC-ROC of ~0.92 and macro F1 of ~0.86 substantially exceed the random baseline, confirming that customer behaviour data genuinely predicts churn at this timescale. The convergence of both interpretability methods on `Lifetime_Value`, `Customer_Service_Calls`, and `Cart_Abandonment_Rate` as the top three features provides directly actionable intelligence: intervention should target customers exhibiting high service contact volume, recent cart abandonment, and declining lifetime value — the combination that the churn-risk profile analysis showed drives predicted churn probability well above the 28.9% base rate.

4.1 Limitations and Future Work

- **Synthetic dataset:** The Kaggle dataset used here is generated rather than drawn from a real business system. Feature correlations, class balance, and even the churn label itself may not reflect real e-commerce dynamics, which limits how far results generalise.

- **Shallow network ceiling:** The Universal Approximation Theorem guarantees representational capacity in theory, but a single hidden layer may still underfit genuinely high-order feature interactions present in real churn datasets. A gradient-boosted tree ensemble (e.g., XGBoost) would likely outperform the shallow network on a real dataset and would maintain interpretability via SHAP values [6].
- **Weight-based importance limitations:** As noted in Section 3.3, this method ignores direction and interactions. SHAP would provide per-prediction, signed feature attributions and should replace weight importance in any production deployment.
- **No temporal validation:** Real churn models should be validated on a future time window (train on months 1–6, test on months 7–8) to assess whether the patterns discovered are stable over time. Cross-sectional train/test splits, as used here, can overestimate real-world performance.
- **Threshold choice sensitivity:** The optimal threshold was selected to maximise F1 but businesses may prefer to maximise recall (to catch as many churners as possible) or precision (to avoid costly false-positive interventions). The final threshold should be tuned against the cost ratio of a missed churner versus a wasted intervention in the specific deployment context.

5 References

[1] F. F. Reichheld, *The loyalty effect: The hidden force behind growth, profits, and lasting value.* Boston, MA: Harvard Business School Press, 1996.

[2] W. Verbeke, K. Dejaeger, D. Martens, J. Hur, and B. Baesens, "New insights into churn prediction in the telecommunication sector: A profit driven data mining approach," *European Journal of Operational Research*, vol. 218, no. 1, pp. 211–229, 2012.

[3] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989.

[4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[5] P. S. Fader, B. G. S. Hardie, and K. L. Lee, ""Counting your customers" the easy way: An alternative to the pareto/NBD model," *Marketing Science*, vol. 24, no. 2, pp. 275–284, 2005.

[6] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in neural information processing systems*, vol. 30, Curran Associates, Inc., 2017.

6 Appendix

6.1 How to Run

```
# Activate virtual environment
source venv/bin/activate

# Run the full pipeline (generates all outputs)
python task1_churn_prediction.py
```

6.2 Output Files

Table 2: Output Files Reference

File	Description
`outputs/01_target_distribution.png`	Target variable bar chart
`outputs/02_correlation_heatmap.png`	Feature correlation matrix
`outputs/03_feature_distributions.png`	Histograms by churn status
`outputs/04_pca_scatter.png`	PCA 2D projection (EDA)
`outputs/05_mutual_information.png`	MI feature ranking
`outputs/06_training_curves.png`	Loss and accuracy curves
`outputs/07_confusion_matrix.png`	Confusion matrix heatmap
`outputs/08_roc_curve.png`	ROC curve
`outputs/09_pr_curve_threshold.png`	Precision-recall curve and F1 vs threshold
`outputs/10_feature_importance.png`	Weight-based and permutation importance